



INSTRUÇÕES

- Esta é uma **atividade individual**, em laboratório, da disciplina “Algoritmos e Estruturas de Dados II”.
- Utilize seu repositório, criado no **GitHub Classroom**, para resolver as questões de programação propostas e enviar suas respostas para correção. **Nenhuma outra forma de entrega da atividade será aceita.**
- O prazo **limite para entrega** é o dia **02/07**, às **12h20**. *Commits* no repositório com data posterior a esse limite serão desconsiderados no momento da correção da atividade.
- Além do **conteúdo do seu repositório de trabalho** e dos **materiais disponíveis no Canvas citados na Tarefa 0**, **não é permitida a consulta a nenhum outro material**, porém a professora está **disponível para esclarecer dúvidas** e debater propostas de soluções vindas dos alunos.
- Sendo uma atividade individual, **espera-se dos alunos comprometimento e ética** na produção de sua solução. Soluções produzidas em conjunto, entregas contendo cópias, mesmo que parciais, em código ou estrutura, e consultas a materiais e ferramentas não permitidos são **faltas graves, resultando em nota 0 para todos os envolvidos** e posterior **notificação à Coordenação do Curso de Engenharia de Software** por desrespeito às normas acadêmicas.

Tema: Busca e relatórios em sistemas de *software*

Temos nosso sistema de comércio de produtos cujos dados são armazenados em estruturas de dados apropriadas para buscas. Nosso sistema já gera alguns relatórios relativos a produtos e pedidos. Nesta última atividade, acrescentaremos ao nosso sistema de comércio uma nova entidade – cliente – e desenvolveremos relatórios sobre clientes e seu histórico de pedidos.

Tarefa 0: (preparação)

- **Adicione uma nova ramificação (branch)** no seu repositório GitHub Classroom correspondente a esta atividade, com o nome **atividade2206**, a partir de seu *branch* anterior. Certifique-se de que as classes **ABB**, **AVL**, **No**, **TabelaHash**, **Entrada**, **Lista** e **Celula** estejam neste novo *branch*.
- Em seguida, adicione à pasta raiz do projeto o arquivo “**clientes.txt**” fornecido no Canvas.

Tarefa 1: (0,5 ponto)

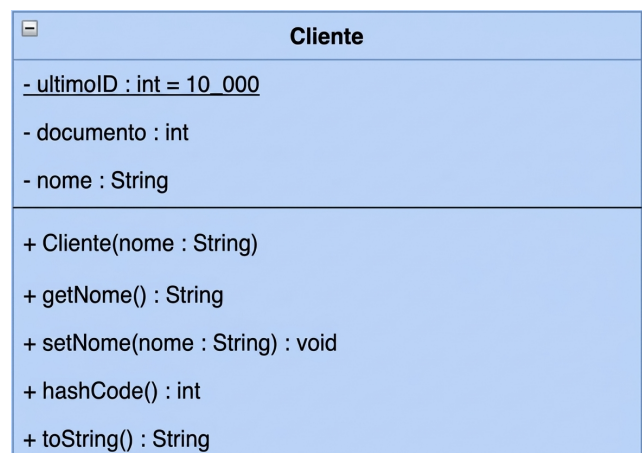
Implemente o código da classe **Cliente**, de acordo com o diagrama de classes UML ao lado.

O construtor da classe cria um novo cliente a partir do nome informado. Em sua implementação, utilize **obrigatoriamente** o método **setNome**. O cliente criado recebe um número de documento gerado sequencialmente a partir do contador estático da classe.

O método **getNome** retorna o nome do cliente; e o método **setNome** atribui ao cliente o nome informado como parâmetro. Esse nome deve conter, pelo menos, duas palavras; caso contrário, a exceção adequada deve ser lançada.

O método **toString** retorna uma representação textual do cliente, incluindo seu nome e documento.

Por fim, o método **hashCode** retorna um código *hash* para o cliente que, neste caso, corresponde a seu documento.



Tarefa 2: (0,5 ponto)

Refatore a classe **Pedido** de forma que, agora, seja possível armazenar o cliente que realizou o pedido. Para isso, adicione um novo atributo à classe e atualize a implementação dos métodos construtor e **toString**.

Tarefa 3: (1 ponto)

Na classe **App**, declare, instancie e inicialize a árvore balanceada AVL em que os clientes serão inseridos. Implemente também o método responsável por ler os dados de um arquivo-texto, cujo nome é fornecido como parâmetro, e retornar uma árvore balanceada AVL de clientes. O arquivo-texto de clientes deve estar no formato:

- N (quantidade de clientes)

- nome do cliente

Considere que haverá uma linha para cada um dos clientes.

O método deve retornar uma árvore com os clientes carregados, ou vazia em caso de problemas na leitura do arquivo.

Tarefa 4: (2,5 pontos)

Na classe **App**, declare e instancie a tabela *hash* que associará cada cliente a seu respectivo histórico de pedidos. Essa tabela *hash* deve ser adequadamente preenchida durante a execução do método **gerarPedido**. Nesse momento, quando um novo pedido for criado, um cliente deve ser selecionado aleatoriamente para ser associado a ele. Adicionalmente, o cliente deve ser vinculado ao seu novo pedido, na tabela *hash* que relaciona um cliente ao seu histórico de pedidos.

Implemente na classe **App** o método **rankingClientes()**, que percorre a tabela *hash* e produz um relatório listando todos os clientes que possuem ao menos dois pedidos, exibindo, para cada um: nome, documento, quantidade de pedidos e valor total acumulado dos pedidos. O relatório deve ser apresentado em ordem decrescente de valor total.

Tarefa 5: (2.5 pontos)

Na classe **App**, implemente o código do método **pedidosDoCliente**, que deve retornar o histórico completo de pedidos do cliente escolhido pelo usuário.

Faça ainda as modificações necessárias no **menu** e no método **main** para que essa opção esteja disponível ao usuário da aplicação.

Além de exibir o histórico completo, o método deve também apresentar um resumo ao final: número total de pedidos do cliente, valor total gasto e valor médio por pedido. Implemente ainda a opção de filtrar os pedidos por valor mínimo, informado pelo usuário: ao selecionar essa opção no menu, o sistema deve solicitar um valor e exibir apenas os pedidos do cliente que o superem.

Instruções e observações:

- O projeto deve estar hospedado na tarefa correspondente do GitHub Classroom. Endereço para aceitar a tarefa: <https://classroom.github.com/a/CF1QPmxa>
- As atividades pontuadas da disciplina podem depender direta ou indiretamente dos códigos desenvolvidos nas aulas. Portanto, é essencial o comprometimento no acompanhamento das atividades semanais.
- Para a correção das atividades pontuadas, serão considerados todos os *commits/pushes* realizados ao longo das semanas, não somente o último com a resposta final do exercício.